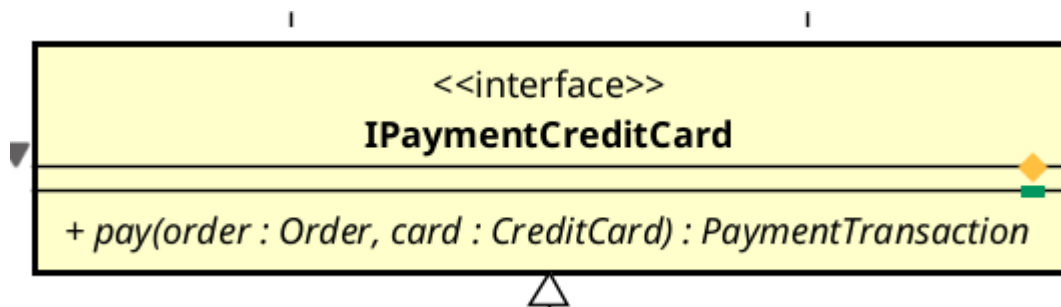


Group 18 - Class Specification

1. Design for class “IPaymentCreditCard”



Description: A general interface defining the contract for credit card payment processing. It decouples the AIMS system from specific payment gateway implementations.

Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	pay	PaymentTransaction	Executes a payment for a specific order using a credit card.

Parameters :

- order: An instance of Order containing total amount and ID
- card: An instance of CreditCard containing card details

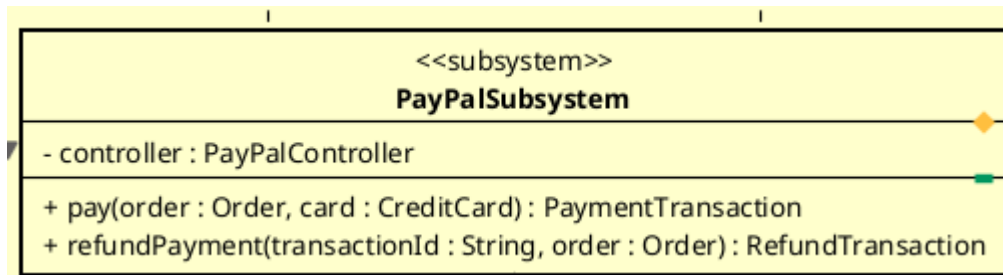
Exceptions:

- PaymentException: Thrown if the transaction fails due to various gateway or network issues.

Methods:

- pay: Takes order details and card information to initiate a transaction via the concrete subsystem implementation.

2. Design for class “PayPalSubsystem”



Description: The Facade class for the PayPal subsystem. It implements the payment and refund interfaces to provide a simplified entry point for the rest of the AIMS application.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	controller	PayPalController	null	The internal controller that handles subsystem logic

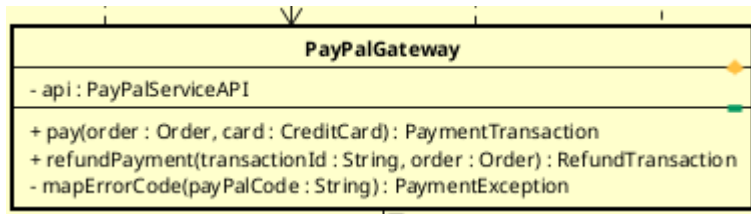
Table 2. Operation design

#	Name	Return type	Description (purpose)
1	pay	PaymentTransaction	Orchestrates the payment process by calling the controller
2	refundPayment	RefundTransaction	Orchestrates the refund process for a specific transaction

Methods:

- Acts as a wrapper; it delegates the pay and refundPayment calls directly to the PayPalController.

3. Design for class “PayPalGateway”



Description: Acts as an Adapter/Gateway that translates domain-specific data (Order, CreditCard) into PayPal-specific API requests and handles error mapping.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	api	PayPalService API	null	The boundary object used to communicate with PayPal's external REST API

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	pay	PaymentTransaction	Coordinates DTO creation, API calling, and response mapping
2	mapErrorCode	PaymentException	Translates PayPal-specific error strings into AIMS Exceptions
3	refundPayment	RefundTransaction	The method for implementing the refund process for customers.

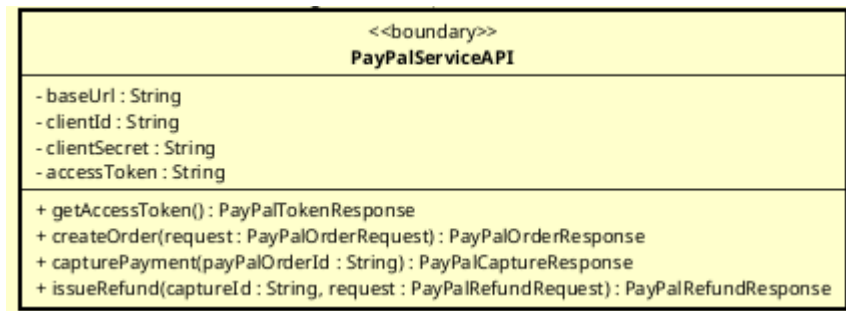
Exceptions:

- InsufficientBalanceException: If PayPal returns "INSTRUMENT_DECLINED".
- PaymentDeclinedException: If the card is rejected by the processor.

Methods:

- pay: Converts Order and CreditCard into a PayPalOrderRequest, calls `api.createOrder`, then `api.capturePayment`, and finally maps the response to a PaymentTransaction.

4. Design for class “PayPalServiceAPI”



Description: A Boundary class responsible for the actual HTTP communication with the PayPal REST API v2.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	baseUrl	String	" https://api-m.sandbox.paypal.com "	The endpoint for PayPal API
2	clientId	String		Unique identifier for the AIMS PayPal account
3	accessToken	String	null	Bearer token for OAuth 2.0 authentication

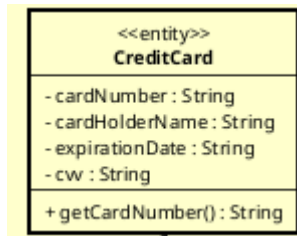
Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getAccessToken	PayPalTokenResponse	Requests a new OAuth2 token using client credentials
2	createOrder	PayPalOrderResponse	Sends a POST request to <code>/v2/checkout/orders</code>
3	capturePayment	PayPalCaptureResponse	Finalizes the payment after the order is created

Methods:

- createOrder: Marshals the PayPalOrderRequest into JSON and executes the HTTP POST request with the current accessToken.

5. Design for class “CreditCard”



Description: An entity class that stores the customer's credit card details required for payment. AIMS does not validate these details; they are passed directly to PayPal for processing.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	cardNumber	String		The 16-digit card number
2	cardHolderName	String		Name printed on the card
3	expirationDate	String		Expiry date (MM/YY)
4	cvv	String		Security code

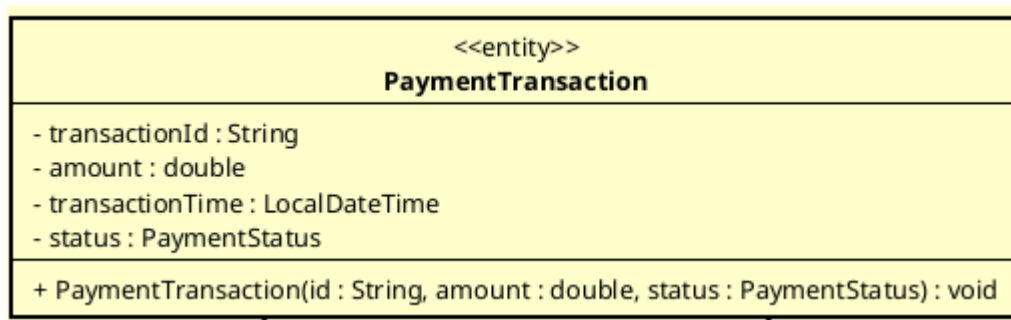
Table 2. Operation design

#	Name	Return type	Description (purpose)
1	getCardNumber	String	Returns the card number for display or processing

Methods:

- **getCardNumber:** Accesses the cardNumber attribute to provide it to the PayPalOrderRequest mapper.

6. Design for class “PaymentTransaction”



Description: An entity class that records the details of a completed or attempted payment transaction.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	transactionId	String		Unique ID returned by PayPal after capture
2	amount	double	0.0	Total amount paid (VAT included)
3	status	PaymentStatus	PENDING	The current state of the payment (COMPLETED, FAILED, PENDING)
4	transactionTime	LocalDateTime	null	The timestamp when the transaction occurred

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PaymentTransaction	(Construct or)	Initializes a transaction record with provided ID and status

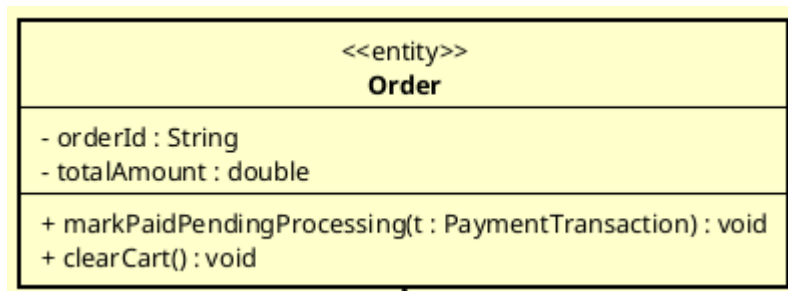
Parameters :

- id: String (No default). The ID from the gateway.
- amount: double (No default). The actual charged amount.
- status: PaymentStatus (No default). The outcome of the payment.

Methods:

- **Constructor:** Assigns parameters to internal attributes and sets transactionTime to the current system time.

7. Design for class “Order”



Description: Represents a customer's order in the AIMS system. It is updated once a payment transaction is successful.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	orderId	String	""	Unique identifier for the order.
2	totalAmount	double	0.0	The sum of product prices (with VAT) and shipping fees.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	markPaidPendingProcessing	void	Updates order status to "pending processing" after payment.

2	clearCart	void	Empties the shopping cart once the order is successfully paid.
---	-----------	------	--

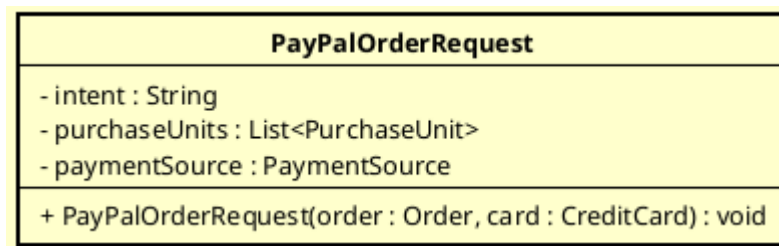
Parameters :

- t: PaymentTransaction. The transaction object used to link payment to the order.

Methods:

- **markPaidPendingProcessing:** Uses the t.status to confirm the payment is COMPLETED before updating the order state to "Pending Processing".

8. Design for class “PayPalOrderRequest”



Description: A Data Transfer Object used to structure the request body for PayPal's "Create Order" API.

Table 1. Sttribute design

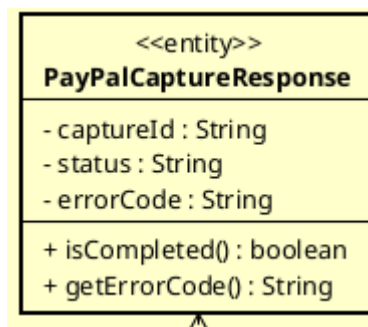
#	Name	Data type	Default value	Description
1	intent	String	"CAPTURE"	The payment intent (CAPTURE or AUTHORIZE).
2	purchaseUnits	List<Purchase Unit>	null	Details of the items and total price.

#	Name	Data type	Default value	Description
3	paymentSource	PaymentSource	null	Contains the CreditCard details for the API.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PayPalOrderRequest	(Construct or)	Maps AIMS domain objects to PayPal DTO structure.

9. Design for class “PayPalCaptureResponse”



Description: An entity class that acts as a Data Transfer Object (DTO). It captures the JSON response from the PayPal "Capture Payment for Order" API endpoint. It contains the final status of the fund transfer and any error codes returned by the processor.

Table 1. Attribute design

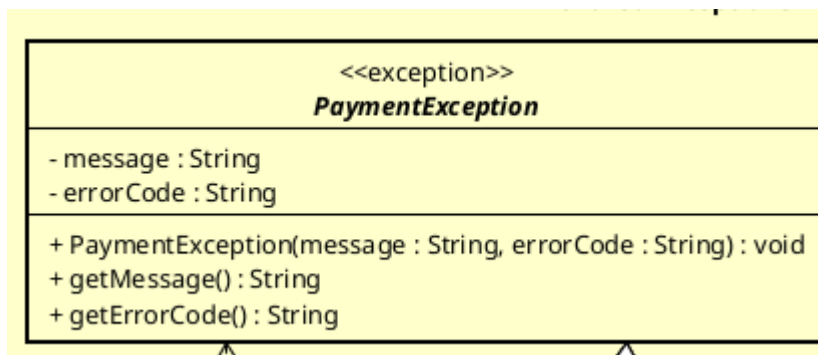
#	Name	Data type	Default value	Description
1	captureId	String	""	The specific ID for the captured funds.
2	status	String	""	PayPal status (e.g., "COMPLETED").

#	Name	Data type	Default value	Description
3	errorCode	String	null	Specific PayPal error code if the capture fails.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	isCompleted	boolean	Returns true if the status is "COMPLETED".
2	getErrorCode	String	Returns the error code for exception mapping.

10. Design for class “PaymentException”



Description: An abstract base class for all payment-related errors in the system.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	message	String	""	Error message for the user.

#	Name	Data type	Default value	Description
2	errorCode	String	""	Internal code for debugging.

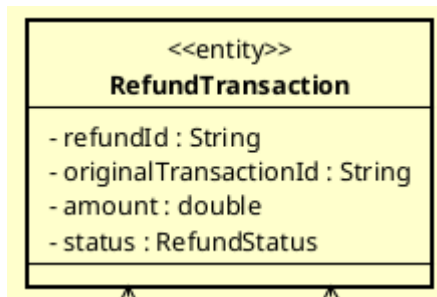
Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	PaymentException	void	Constructor to initialize the exception with a specific message and error code.
2	getMessage	String	Returns the human-readable error message associated with the exception.
3	getErrorCode	String	Returns the internal error code associated with the exception.

Exceptions:

- InsufficientBalanceException: If PayPal returns "INSTRUMENT_DECLINED".
- PaymentDeclinedException: If the transaction is rejected by the bank.
- PaymentTimeoutException: If the API call fails to respond within the required time.

11. Design for class “RefundTransaction”



Description: Records information about a refund issued to a customer.

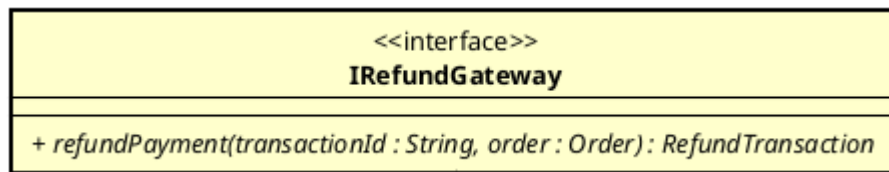
Table 1. Attribute design

#	Name	Data type	Default value	Description
1	refundId	String	""	The unique ID for the refund transaction.
2	originalTransactionId	String	""	Link to the original payment ID.
3	amount	double	0.0	The amount refunded.
4	status	RefundStatus	FAILED	Result: SUCCESS or FAILED.

Table 2. Operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

12. Design for class “IRefundGateway”



Description: An interface that defines the contract for refunding payments. It allows the AIMS system to process refunds through any payment gateway that implements this interface.

Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	refundPayment	RefundTransaction	Processes a refund for a previously captured payment.

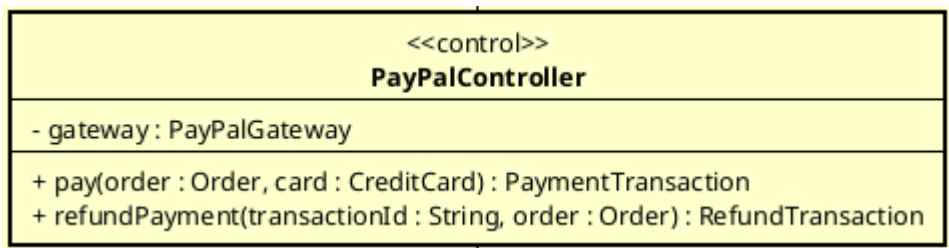
Parameters :

- transactionId: String. Default: none. The ID of the original payment transaction to be refunded.
- order: Order. Default: none. The order object associated with the refund.

Methods:

- refundPayment: Uses the transactionId to identify the funds at the gateway and the order's total amount to determine the refund value.

13. Design for class “PayPalController”



Description: A control class that coordinates the logic within the PayPal subsystem, acting as a bridge between the Subsystem Facade and the Gateway.

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	gateway	PayPalGateway	null	The adapter used to communicate with the PayPal API.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	pay	PaymentTransaction	Coordinates the payment flow.
2	refundPayment	RefundTransaction	Coordinates the refund flow.

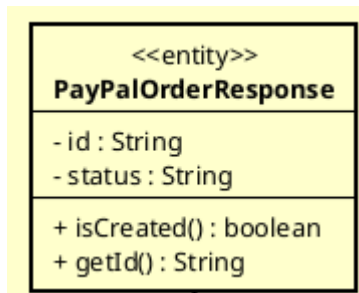
Parameters :

- order: Order. Current order to pay/refund.
- card: CreditCard. Required for payment operations.
- transactionId: String. Required for refund operations.

Methods:

- pay: Invokes gateway.pay(order, card) and handles high-level subsystem logic.
- refundPayment: Invokes gateway.refundPayment(transactionId, order).

14. Design for class “PayPalOrderResponse”



Description: An entity class representing the response received from PayPal after an order creation request.

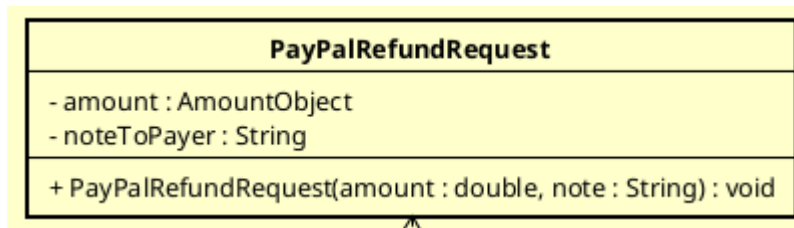
Table 1. Attribute design

#	Name	Data type	Default value	Description
1	id	String	""	The Order ID generated by PayPal.
2	status	String	""	The status of the created order (e.g., "CREATED").

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	isCreated	boolean	Checks if the order was successfully created on PayPal's side.
2	getId	String	Returns the PayPal Order ID.

15. Design for class “PayPalRefundRequest”



Description: A DTO (Data Transfer Object) used to encapsulate data for the PayPal Refund API request.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	amount	AmountObject	null	Object containing value and currency code.
2	noteToPayer	String	""	An optional message sent to the customer regarding the refund.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PayPalRefundRequest	(Construct or)	Creates a request object with specific amount and note.

Parameters :

- amount: double. The numeric value to refund.
- note: String. The reason or note for the refund.

16. Design for class “PayPalRefundResponse”

PayPalRefundResponse
- id : String - status : String
+ isRefunded() : boolean

Description: Represents the result returned by the PayPal API after a refund attempt.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	id	String	""	The unique ID for the refund transaction.
2	status	String	""	Current status (e.g., "COMPLETED", "CANCELLED").

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	isRefunded	boolean	Returns true if the status is "COMPLETED".

17. Design for class “PayPalTokenResponse”

PayPalTokenResponse
- accessToken : String - tokenType : String - expiresIn : int
+ isValid() : boolean

Description: Holds the OAuth 2.0 access token and its metadata required for authenticated API calls.

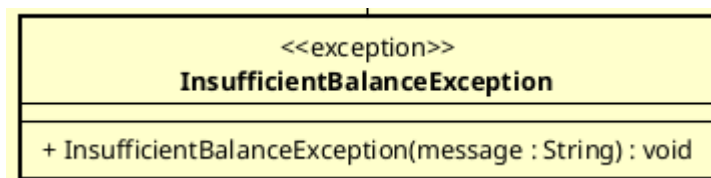
Table 1. Attribute design

#	Name	Data type	Default value	Description
1	accessToken	String	""	The token string used in Authorization headers.
2	tokenType	String	"Bearer"	The type of token.
3	expiresIn	int	0	Token validity duration in seconds.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	isValid	boolean	Checks if the token is present and not expired.

18. Design for class “InsufficientBalanceException”



Description: A specialized exception thrown when the customer's credit card has insufficient funds to complete the transaction.

Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2.Operation design

#	Name	Return type	Description (purpose)
1	InsufficientBalanceException	(Construct or)	Creates an exception with a specific error message.

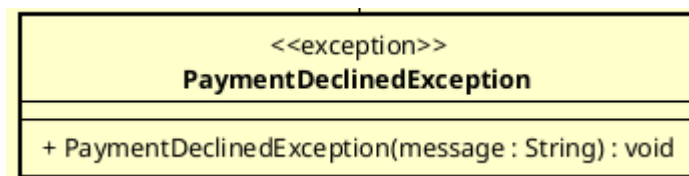
Parameters :

- message: String. Default: "Your card has insufficient balance."

Methods:

- Calls super(message, "INSUFFICIENT_FUNDS") to initialize the base PaymentException.

19. Design for class “PaymentDeclinedException”



Description: Thrown when the payment is rejected by the issuing bank for reasons other than balance (e.g., card blocked, expired, or fraud suspicion).

Table 1. Attribute design

#	Name	Data type	Default value	Description
---	------	-----------	---------------	-------------

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PaymentDeclinedException	(Construct or)	Creates an exception for declined payments.

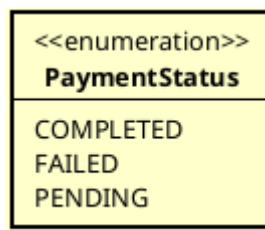
Parameters :

- message: String. Description of why the card was declined.

Methods:

- Calls super(message, "PAYMENT_DECLINED").

20. Design for class “PaymentStatus”

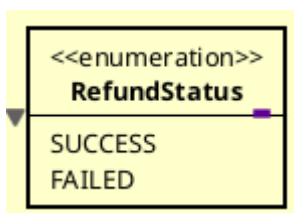


Description: Defines the possible states of a payment transaction within the AIMS system.

Table 1.Values

Value	Description
COMPLETED	The funds have been successfully captured from the customer.
FAILED	The payment process stopped due to an error or declination.
PENDING	The payment has been initiated but not yet finalized.

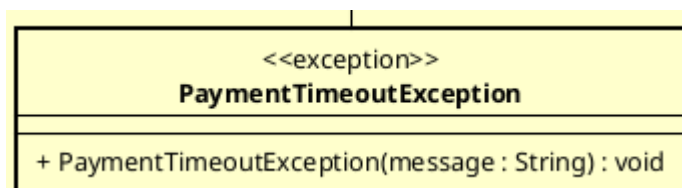
21. Design for class “RefundStatus”



Description: Defines the outcome of a refund request.

Value	Description
SUCCESS	The money has been successfully sent back to the customer.
FAILED	The refund could not be processed by the gateway.

22. Design for class “PaymentTimeoutException”



Description: It is thrown by the PayPalGateway when a network request to the external PayPal API fails to receive a response within the system's defined time limits.

Table 1. Attribute design

#	Name	Data type	Default value	Description
1	message	String	(Inherited)	A descriptive message explaining the timeout error to the user.
2	errorCode	String	(Inherited)	A specific code used to identify the timeout failure internally.

Table 2. Operation design

#	Name	Return type	Description (purpose)
1	PaymentTimeoutException	void	Constructor to initialize the exception with a specific error message.

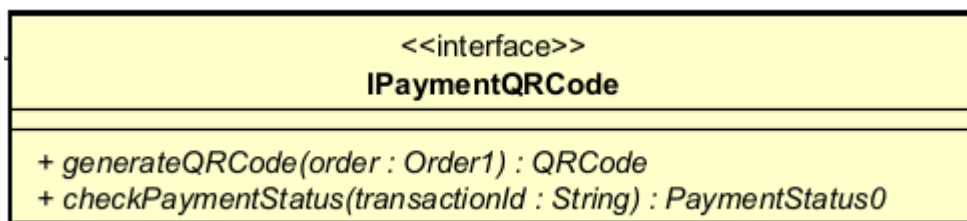
Parameters :

- **message**: String. Default value: none. The error message providing details about the connection or response timeout.

Exceptions:

- This class is an exception itself and does not throw other exceptions during instantiation.

23. Design for class “IPaymentQRCode”



Description: Interface that defines the common contract for QR-code-based payment. It allows the application layer (PaymentService) to generate QR codes and query transaction status without coupling to any specific payment gateway (VietQR, ZaloPay, etc.). All implementations are responsible for handling HTTP communication, authentication, and internal response mapping

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
—	—	—	—	Interface carries no state

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
---	------	-------------	-----------------------

1	generateQRCode	QRCode	Creates a payment QR code from the supplied order information. Validates input, delegates to the subsystem, and returns a QRCode entity ready for display.
2	checkPaymentStatus	PaymentStatus	Queries the status of a transaction by its transactionId. Returns PENDING, SUCCESS, or FAILED.

Parameters :

Parameter	Type	Constraint
order	Order	Non-null; must carry a non-null orderId, an amount greater than zero, and a payment content string.
transactionId	String	Non-null, non-blank transaction identifier.

Exceptions:

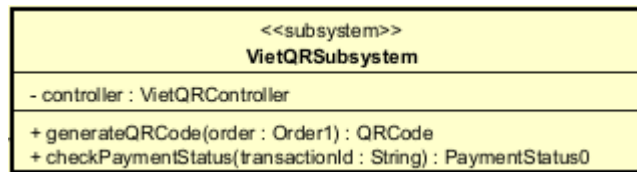
Exception	Condition
PaymentException	Base exception; raised when the operation fails for any payment-related reason.
PaymentTimeoutException	Raised when authentication or an API call exceeds the allowed timeout.
InvalidTransactionException	Raised when transactionId is null, blank, or does not correspond to a known transaction.

Methods:

- generateQRCode: requires a non-null order; delegates to the subsystem realization and returns the resulting QRCode.
- checkPaymentStatus: requires a non-blank transactionId; delegates to the subsystem and returns the corresponding PaymentStatus.

Other diagrams

24. Design for class “VietQRSubsystem”



Description: The sole externally visible façade of the VietQR package. Realises IPaymentQRCode and delegates all processing to VietQRController. All internal classes (Controller, Gateway, ServiceAPI) must remain hidden; external callers interact exclusively through this interface.

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	controller	VietQRController	null	Internal controller; not accessible from outside the subsystem.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	generateQRCode	QRCode	Validates the order and delegates to the controller. Propagates any exception without wrapping.
2	checkPaymentStatus	PaymentStatus	Validates the transactionId and delegates to the controller. Propagates any exception without wrapping.

Parameters :

Parameter	Type	Constraint
order	Order	Non-null; must have amount > 0 and a non-null orderId.
transactionId	String	Non-null, non-blank.

Exceptions:

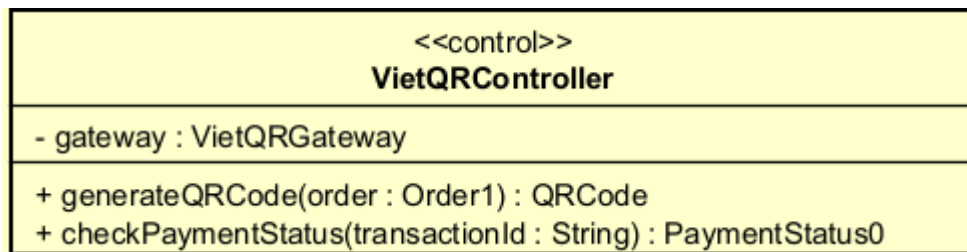
Exception	Condition
PaymentException	Raised when order is null.

InvalidTransactionException	Raised when transactionId is null or blank.
(propagated)	All exceptions originating from the controller are surfaced to the caller unchanged.

Methods:

- generateQRCode: guards against a null order, then delegates to the controller. All PaymentException subclasses propagate to the caller without wrapping.
- checkPaymentStatus: guards against a null or blank transactionId, then delegates to the controller. All PaymentException subclasses propagate to the caller without wrapping.

25. Design for class “VietQRController”



Description: Coordinates the internal payment workflow within the subsystem. Receives requests from VietQRSubsystem and delegates all API communication to VietQRGateway. Does not issue HTTP calls directly and does not construct domain entities.

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	gateway	VietQRGateway	null	Gateway used for all API interactions; injected via constructor.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	generateQRCode	QRCode	Receives an order from the subsystem, delegates to the gateway, and returns the result.
2	checkPaymentStatus	PaymentStatus	Receives a transactionId from the subsystem, delegates to the gateway, and returns the result.

Parameters :

Parameter	Type	Constraint
order	Order	Non-null Order supplied by VietQRSubsystem.
transactionId	String	Non-blank transaction identifier.

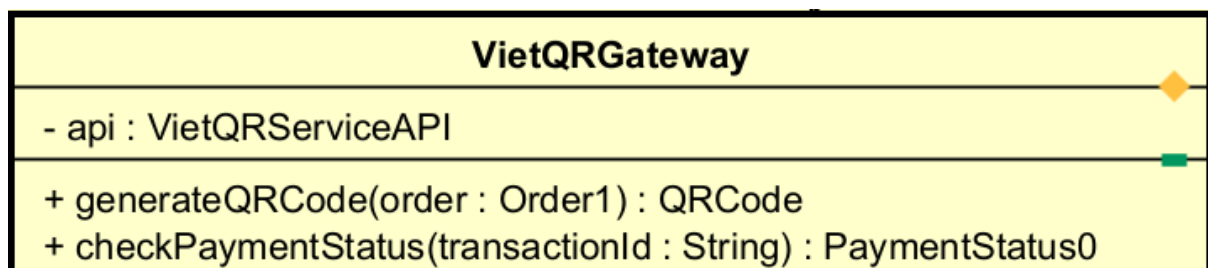
Exceptions:

Exception	Condition
InvalidQRCodeException	Propagated from gateway when the QR code received from the API is null or empty.
PaymentTimeoutException	Propagated from gateway when authentication or an API call times out.
InvalidTransactionException	Propagated from gateway when the transaction cannot be found.

Methods:

- generateQRCode: delegates directly to the gateway and returns the resulting QRCode. All PaymentException subclasses are propagated to the subsystem.
- checkPaymentStatus: delegates directly to the gateway and returns the resulting PaymentStatus. All PaymentException subclasses are propagated to the subsystem.

26. Design for class “VietQRGateway”



Description: Adapter between the controller and VietQRServiceAPI. Orchestrates the API call sequence (build request → authenticate → call endpoint) and maps VietQRResponse objects to domain objects (QRCode or PaymentStatus). The controller is shielded from HTTP concerns and response structure details.

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	api	VietQRService API	null	HTTP client for all communication with the VietQR API; injected via constructor.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	generateQRCode	QRCode	Builds the request, authenticates, calls the QR-generate endpoint, and maps the response to a QRCode entity.
2	checkPaymentStatus	PaymentStatus	Authenticates, calls the transaction-status endpoint, and maps response.status to a PaymentStatus enum value.

Parameters :

Parameter	Type	Constraint
order	Order	Non-null; amount > 0, orderId non-null.
transactionId	String	Non-blank transaction identifier.

Exceptions:

Exception	Condition
PaymentTimeoutException	Raised when the token response is null or the token string is blank (authentication failure).
InvalidQRCodeException	Raised when VietQRResponse.qrCode is null or blank.
InvalidTransactionException	Raised when VietQR returns HTTP 404 for the transaction query.
PaymentException	Raised for any other HTTP error.

Methods:

- generateQRCode: requests a QR payload by first obtaining an access token via the API, then calling the QR-generation endpoint. Validates that the token and the

returned QR code are both non-blank before constructing and returning a new QRCode entity.

- checkPaymentStatus: obtains an access token, queries the transaction-status endpoint, and maps the string status field in the response to the corresponding PaymentStatus constant. Returns PENDING if the response is null or the status is unrecognised.

27. Design for class “VietQRServiceAPI”

VietQRServiceAPI	
- baseUrl : String - clientId : String - apiKey : String - accessToken : String	
+ getAccessToken(username : String, password : String) : TokenResponse + buildQRRequest(order : Order1) : VietQRRequest + generateQRCode(request : VietQRRequest) : VietQRResponse + checkTransactionStatus(transactionId : String) : VietQRResponse	

Description: Pure HTTP client for the VietQR API. Manages the access token, constructs request objects, dispatches HTTP calls, and parses JSON responses into entity objects. Contains no business logic.

Table 1. Example of attribute design

#	Name	Data type	Default value	Description
1	baseUrl	String	""	Base URL of the VietQR REST API.
2	clientId	String	""	Client ID credential for authentication.
3	apiKey	String	""	API key credential for authentication.
4	accessToken	String	""	Bearer token stored after a successful getAccessToken call.

Table 2. Example of operation design

#	Name	Return type	Description (purpose)
1	getAccessToken	TokenResponse	Sends a POST to the auth endpoint and stores the token.

2	buildQRRequest	VietQRRequest	Constructs a VietQRRequest from an Order. Issues no HTTP call.
3	generateQRCode	VietQRResponse	Sends a POST to the QR-generation endpoint and parses the JSON response.
4	checkTransactionStatus	VietQRResponse	Sends a GET to the transaction-status endpoint and parses the JSON response.

Parameters :

Parameter	Type	Constraint
username	String	Authentication username (clientId-based).
password	String	Authentication password (apiKey-based).
order	Order	Must have amount > 0 and a non-null orderId.
request	VietQRRequest	Pre-built object produced by buildQRRequest().
transactionId	String	Non-blank VietQR transaction identifier.

Exceptions:

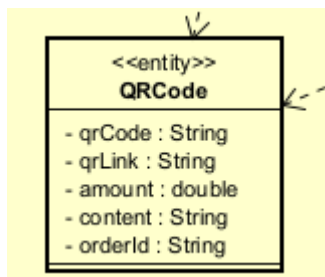
Exception	Condition
PaymentException	Raised when the HTTP status is not 200 (getAccessToken, generateQRCode).
PaymentException	Raised in buildQRRequest when order.amount ≤ 0 or order.orderId is null.
InvalidTransactionException	Raised in checkTransactionStatus when VietQR returns HTTP 404.
PaymentException	Raised in checkTransactionStatus for any other HTTP error.

Methods:

- getAccessToken: posts credentials to the auth endpoint, stores the resulting bearer token, and returns a TokenResponse. Raises PaymentException if the response status is not 200.
- buildQRRequest: Validates order fields and constructs a VietQRRequest from order data and static configuration (bank code, account, holder name). Does not communicate with the network.

- generateQRCode: Attaches the stored bearer token as an Authorization header, serialises the request to JSON, and posts to the QR-generation endpoint. Returns the parsed VietQRResponse; raises PaymentException on a non-200 status.
- checkTransactionStatus: Attaches the stored bearer token and issues a GET to the transaction endpoint. Returns the parsed VietQRResponse; raises InvalidTransactionException on HTTP 404, or PaymentException for other HTTP errors.

28. Design for class “QRCode”



Description: Immutable domain entity representing the QR code returned to the caller. Carries all information required to display a payment QR code to the end user.

Table 1. Example of attribute design

#	Name	Data Type	Default	Description
1	qrCode	String	""	Base64-encoded QR code string.
2	qrLink	String	""	Direct URL to the QR image (used in an element).
3	amount	double	0.0	Payment amount in VND.
4	content	String	""	Payment content embedded in the QR.
5	orderId	String	""	Order ID linking the QR to the transaction.

Table 2. Example of operation design

#	Name	Return Type	Description
1	QRCode	void (constructor)	Creates the entity; validates that qrCode and qrLink are non-blank before assigning all fields.

Parameters :

- qrCode : String, Non-null, non-blank.

- qrLink : String, Non-null, non-blank.
- amount : double, Payment amount in VND.
- content : String, Payment description.
- orderId : String, Order identifier.

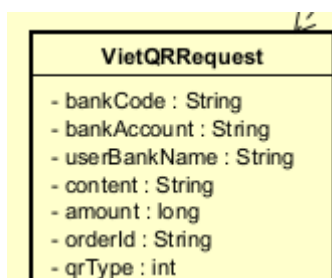
Exceptions:

- IllegalArgumentException Raised by the constructor when qrCode is null or blank
- IllegalArgumentException Raised by the constructor when qrLink is null or blank

Methods:

- Constructor : Validates that qrCode and qrLink are each non-null and non-blank, then assigns all fields. The object is immutable after construction.

29. Design for class “VietQRRequest”



Description: Data Transfer Object serialised to a JSON body when calling the VietQR QR-generation endpoint. Fields map directly to the VietQR API contract. Created exclusively by VietQRServiceAPI.buildQRRequest().

Table 1. Example of attribute design

#	Name	Data Type	Default	Description
1	bankCode	String	""	VietQR bank code (e.g. "VCB", "TCB").
2	bankAccount	String	""	Destination account number.
3	userBankName	String	""	Account holder name.
4	content	String	""	Payment description (max 50 characters per VietQR spec).
5	amount	long	0	Payment amount in VND (whole number, no decimals).
6	orderId	String	""	Order ID for reconciliation.
7	qrType	int	0	0 = dynamic QR, 1 = static QR.

Table 2. Example of operation design

#	Name	Return Type	Description
1	VietQRRequest	void (constructor)	All-args constructor; assigns all fields.

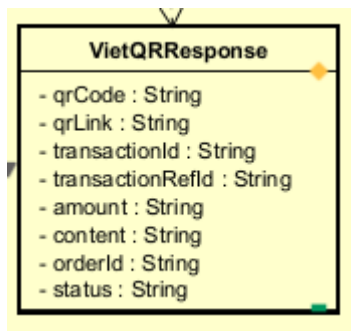
Parameters :

- bankCode, bankAccount, userBankName, content : String
- amount : long
- orderId : String
- qrType : int

Methods:

- Constructor : Directly assigns all supplied parameters to the corresponding fields. No validation is performed; validation is the responsibility of VietQRServiceAPI.buildQRRequest().

30. Design for class “VietQRResponse”



Description: Holds the parsed JSON response from the VietQR API, covering both QR generation and transaction-status queries. VietQRGateway reads this object and maps it to the appropriate domain object (QRCode or PaymentStatus).

Table 1. Example of attribute design

#	Name	Data Type	Default	Description
1	qrCode	String	""	QR code string (present in generateQRCode responses only).
2	qrLink	String	""	QR image URL (present in generateQRCode responses only).
3	transactionId	String	""	VietQR-internal transaction ID.

4	transactionRefId	String	""	Merchant-side reference ID for reconciliation.
5	amount	String	""	Amount as a string per API contract (requires parsing before use).
6	content	String	""	Payment description echoed from the request.
7	orderId	String	""	Order ID echoed from the request.
8	status	String	""	Transaction status: "SUCCESS" "FAILED" "PENDING".

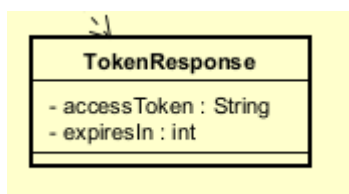
Table 2. Example of operation design

#	Name	Return Type	Description
1	VietQRResponse	void (constructor)	No-args constructor for JSON deserialisation (setter-based).
2	isSuccess	boolean	Returns true when status equals "SUCCESS". Convenience method for VietQRGateway.

Methods:

- isSuccess : Returns true if and only if the status field equals the string "SUCCESS".

31. Design for class “TokenResponse”



Description: Carries the bearer token returned by the VietQR auth endpoint. Used by VietQRGateway to confirm that authentication succeeded before issuing subsequent API calls.

Table 1. Example of attribute design

#	Name	Data Type	Default	Description
1	accessToken	String	""	Bearer token used in the Authorization header.
2	expiresIn	int	0	Token lifetime in seconds (e.g. 3600).

Table 2. Example of operation design

#	Name	Return Type	Description
1	TokenResponse	void (constructor)	Constructor accepting accessToken and expiresIn.
2	isValid	boolean	Returns true when accessToken is non-null and non-blank.
3	getAccessToken	String	Returns the accessToken value. Used by VietQRServiceAPI when building the Authorization header.

-

Methods:

- isValid : Returns true if and only if accessToken is non-null and non-blank.
- getAccessToken : Returns the stored accessToken string.